IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute Of Arts Science Commerce & Management

PROJECT – LOW LEVEL DESIGN

ON

MULTI-CLOUD HYBRID STRATEGY FOR HEALTHCARE SYSTEM

Student Name: Muhammad Razeen TC

Registration no: 23BCAICD077

Campus ID: 25295

Project Code: PRJN26-83

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Sindhu

Table of Contents:

1.Introduction._____ 4

1.1. Scope of the Document.
1.2. Intended Audience.
1.3. System Overview.

2.Detailed System Architecture._____5-6

2.1. Module Description.

3.Database Design (Detailed)._____ 6-8

3.1. Tables and Attributes.
3.2. Relationships.
3.3. Schedule Model.
3.4. Booking Model.
3.5. Prescription Model.
3.6. Complaint Model.

4.Functional Design._____ 8

4.1. Backup Workflow.
4.2. Restore Workflow.

5.Function-Level Design._____ 9-10

5.1. Backup Function.
5.2. Restore Function.

6.Error Handling Mechanism._____ 10-11

7.Security Design._____ 11

8.Scheduler Design._____ 12

9.Logging Mechanism._____ 12

10.Non-Functional Implementation._____ 12

11.Failover Mechanism._____ 12-13

12 .References._____ 13

13 .Detailed Module Interaction_____13-14

13.1. Interaction Between Modules
13.2. Data Flow Integration

14. Backup Strategy (Detailed Explanation) _____14-15

14.1. Multi-Source Backup
14.2. Backup Process Details
14.3. Backup Optimization

1. Introduction

1.1 Scope of the Document

This document provides the **Low-Level Design (LLD)** of the system, describing:

- Detailed module design
- Class-level structure
- Function-level logic
- Database schema
- Backup and restore workflows

It focuses on **implementation-level details** of the system.

1.2 Intended Audience

- Developers
- Project evaluators
- System implementers
- DevOps engineers

1.3 System Overview

The system is a **Django-based Healthcare Application** integrated with a **Multi-Cloud Backup System**.

The LLD explains:

- How each module works internally
- How backup and restore are implemented

- How failover is handled

---

2. System Architecture (Detailed)

2.1 Module Breakdown

1. Authentication Module

- Handles login/logout
- Uses Django authentication

---

2. Doctor Module

- Add doctor
- Manage schedules
- Upload prescriptions

---

3. User Module

- Registration
- Booking appointments
- View prescriptions

---

4. Booking Module

- Connects user and doctor
- Stores appointment data

---

5. Prescription Module

- Stores medical data
- **Triggers backup system**

---

6. Backup Module

- Database backup
- Media backup
- Cloud upload
- Failover handling

---

7. Restore Module

- Fetch backup
- Restore database
- Failover logic

---

3. Class Design (Models)

3.1 Doctor Model

- id
- name
- phone
- email
- department
- qualification
- location details

3.2 User Model

- id
- name
- email
- phone
- address details

3.3 Schedule Model

- doctor_id (FK)
- date
- from_time
- to_time

3.4 Booking Model

- schedule_id (FK)
- user_id (FK)
- date
- status

3.5 Prescription Model

- booking_id (FK)
- date
- prescription
- findings

3.6 Complaint Model

- user_id (FK)
- complaint
- date
- status

---

4. Sequence Flow

4.1 Backup Trigger Flow

1. Doctor uploads prescription
2. `post_save` signal is triggered
3. `trigger_backup_async()` is called
4. Backup runs in background thread
5. Database backup created
6. Media backup created
7. Upload to cloud attempted
8. If failure → fallback to local

4.2 Restore Flow

1. Restore command executed
2. Check local backup
3. If not found → check cloud
4. Download latest backup
5. Restore database

---

5. Function-Level Design

5.1 Backup Functions

`run_full_backup()`

- Controls complete backup process

`backup_database()`

- Uses `mysqldump`
- Saves `.sql` file

`backup_media()`

- Compress media folder
- Save `.zip` file

`_upload_drive()`

- Upload file to Google Drive

`_upload_with_fallback()`

- Try cloud upload
- If fail → keep local

5.2 Restore Functions

`run_restore()`

- Main restore controller

`get_latest_local()`

- Fetch latest local backup

`download_latest_from_drive()`

- Download backup from cloud

`restore_db()`

- Restore MySQL database

---

6. Database Design

Tables Overview

Table Name	Description
doctor	Stores doctor info
user	Stores patient info
schedule	Doctor availability
booking	Appointment data

prescription	Medical records
complaint	User complaints

Relationships

- doctor → schedule (1:N)
- user → booking (1:N)
- booking → prescription (1:1)

---

7. Error Handling Design

Backup Errors:

- mysqldump failure
- cloud upload failure

Handling:

- Log error
- Remove corrupted files
- fallback to local backup

Restore Errors:

- Missing backup file
- MySQL restore failure

Handling:

- Try alternative source (cloud)
- Display error logs

8. Security Design

- Secure login authentication
- OAuth for Google Drive
- MySQL secure config file
- Role-based access control

---

9. Scheduler Design

- Uses APScheduler
- Runs daily backup
- Configurable time

---

10. Logging Design

- Uses Python `logging` module
- Logs:
  - Backup success
  - Backup failure
  - Restore status

---

11. Failover Mechanism

Backup Failover:

- Try cloud upload
- If fails → keep local

Restore Failover:

- Try local
- If fails → use cloud

12.References

[1] Django Framework – https://docs.djangoproject.com/

[2] Python Programming Language – https://docs.python.org/3/

[3] MySQL Database (mysqldump utility) – https://dev.mysql.com/doc/

[4] PyDrive2 Library (Google Drive Integration) – https://docs.iterative.ai/PyDrive2/

[5] APScheduler (Background Scheduler) – https://apscheduler.readthedocs.io/

13. Detailed Module Interaction

The system follows a modular architecture where each module interacts with others in a structured manner to ensure smooth workflow execution.

13.1 Interaction Between Modules

- The **User Module** initiates the process by booking an appointment.
- The **Booking Module** stores and manages appointment details.
- The **Admin Module** verifies and updates booking status (accepted/rejected).
- The **Doctor Module** accesses approved bookings and uploads prescriptions.
- The **Prescription Module** stores medical data and triggers backup.
- The **Backup Module** performs automatic backup operations.
- The **Restore Module** ensures data recovery when required.

This interaction ensures **data consistency and proper flow of information** across the system.

13.2 Data Flow Integration

The data flows through the system in the following sequence:

1. Input is received from the user interface (forms).
2. Data is processed in Django views.
3. Processed data is stored in the database using models.
4. Important actions trigger background processes (backup).
5. Backup data is stored locally and on cloud storage.

This layered flow ensures **separation of concerns and maintainability**.

14. Backup Strategy (Detailed Explanation)

The backup strategy is designed to ensure **data reliability and fault tolerance**.

14.1 Multi-Source Backup

The system stores backup data in:

- Local storage (system directory)
- Cloud storage (Google Drive)

This ensures redundancy and protects against data loss.

14.2 Backup Process Details

- Database backup is created using `mysqldump`
- Media files are compressed into `.zip`
- Files are stored locally first
- Then uploaded to cloud storage

This approach ensures **backup availability even if cloud upload fails**.

14.3 Backup Optimization

- Only the latest backups are retained
- Old backups are automatically deleted
- Backup runs in background thread (non-blocking)

This improves **performance and storage efficiency**.